

Implementing a Full DevSecOps CI/CD Pipeline for Microservices on Kubernetes

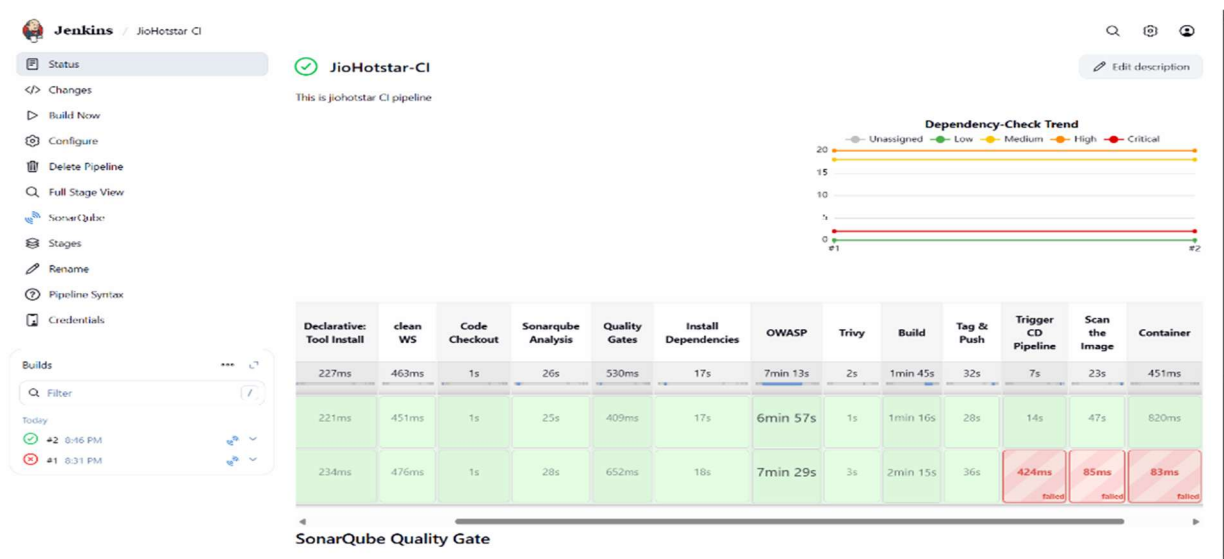
This project demonstrates the complete end-to-end automation of microservice deployment using a modern DevOps toolchain. The architecture integrates Continuous Integration (CI) and Continuous Deployment (CD) pipelines built with Jenkins, SonarQube, Trivy, ArgoCD, and Kubernetes. Each component ensures that source code from GitHub is automatically built, scanned, tested, containerized, and deployed onto Kubernetes clusters with continuous monitoring via Prometheus and Grafana. This approach enhances deployment speed, quality assurance, and system observability while reducing manual intervention.

Tech stack used in this project:

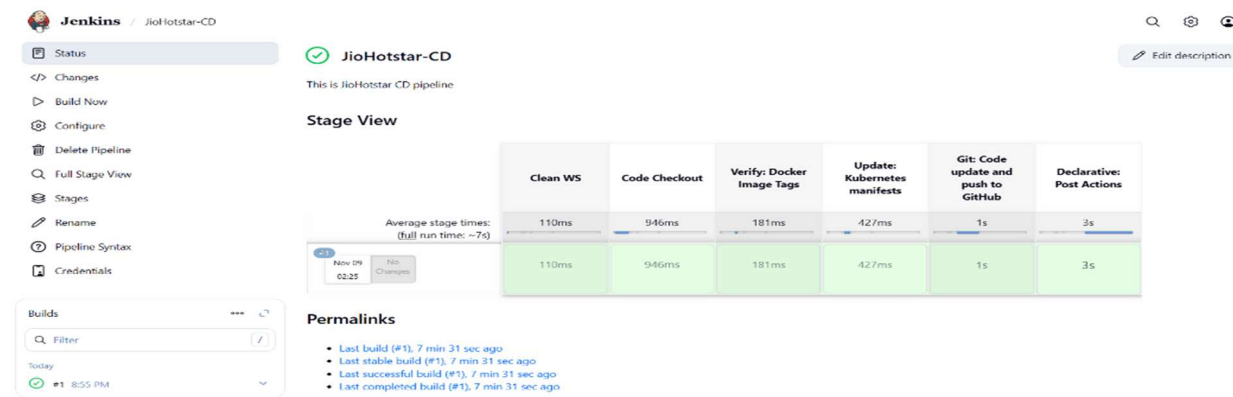
- GitHub (Code)
- Docker (Containerization)
- Jenkins (CI)
- OWASP (Dependency check)
- SonarQube (Quality)
- Trivy (Filesystem Scan)
- ArgoCD (CD)
- AWS EKS (Kubernetes)
- Helm (Monitoring using grafana and prometheus)

How pipeline will look after deployment:

- CI pipeline to build and push



- CD pipeline to update application version



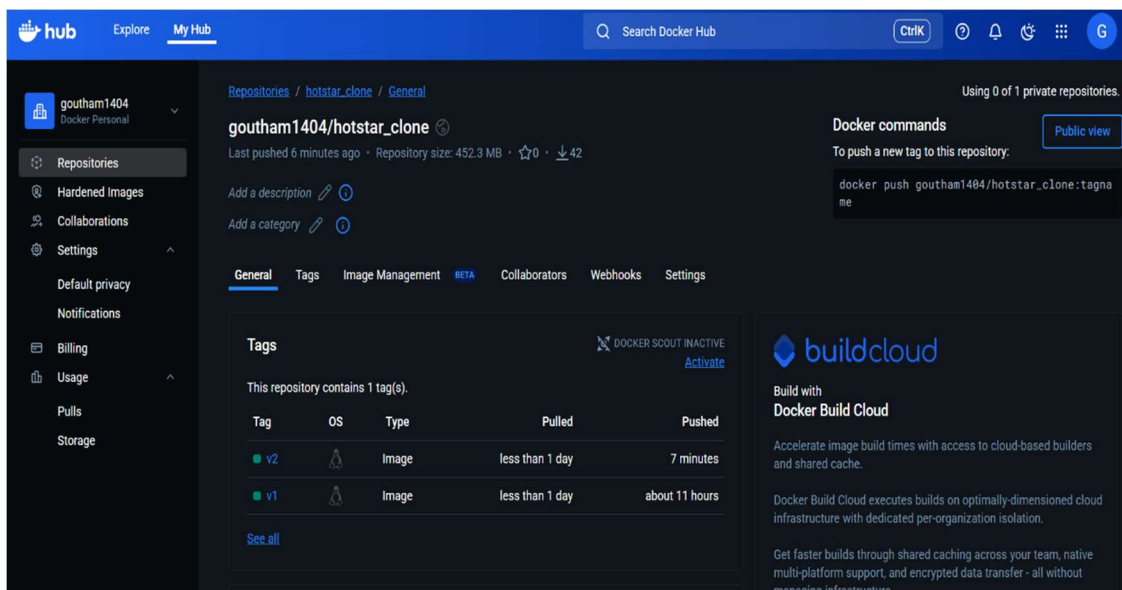
The Jenkins interface shows the 'JioHotstar-CD' pipeline. The left sidebar contains navigation links: Status, Changes, Build Now, Configure, Delete Pipeline, Full Stage View, Stages, Rename, Pipeline Syntax, and Credentials. The main area displays the 'Stage View' with a table of stages and their durations.

Stage	Clean WS	Code Checkout	Verify: Docker Image Tags	Update: Kubernetes manifests	Git: Code update and push to GitHub	Declarative: Post Actions
Average stage times: (full run time: ~7s)	110ms	946ms	181ms	427ms	1s	2s
Nov 09 02:25	110ms	946ms	181ms	427ms	1s	3s

Below the table, the 'Permalinks' section lists build details:

- Last build (#1), 7 min 31 sec ago
- Last stable build (#1), 7 min 31 sec ago
- Last successful build (#1), 7 min 31 sec ago
- Last completed build (#1), 7 min 31 sec ago

- Docker Image after tag & push in JioHotstar-CI pipeline

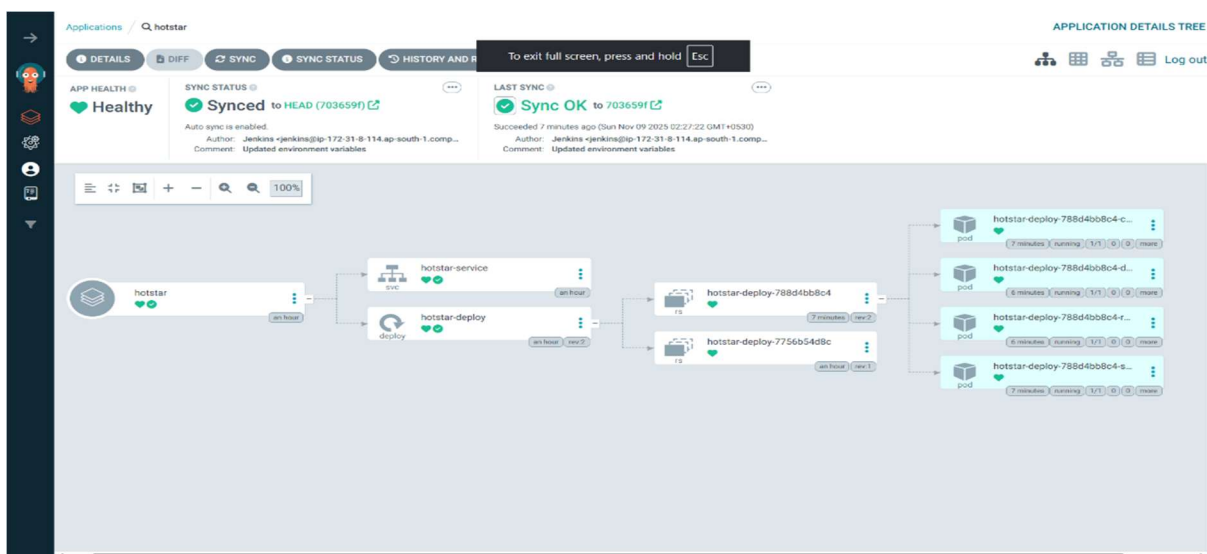


The Docker Hub interface shows the repository 'goutham1404/hotstar_clone'. The left sidebar includes links for Repositories, Hardened Images, Collaborations, Settings, Billing, Usage, Pulls, and Storage. The main area displays the repository details, including the 'Tags' section.

Tag	OS	Type	Pulled	Pushed
v2	linux	Image	less than 1 day	7 minutes
v1	linux	Image	less than 1 day	about 11 hours

The 'Tags' section also includes a 'See all' link and a 'DOCKER SCOUT INACTIVE' button.

- ArgoCD application for deployment on EKS



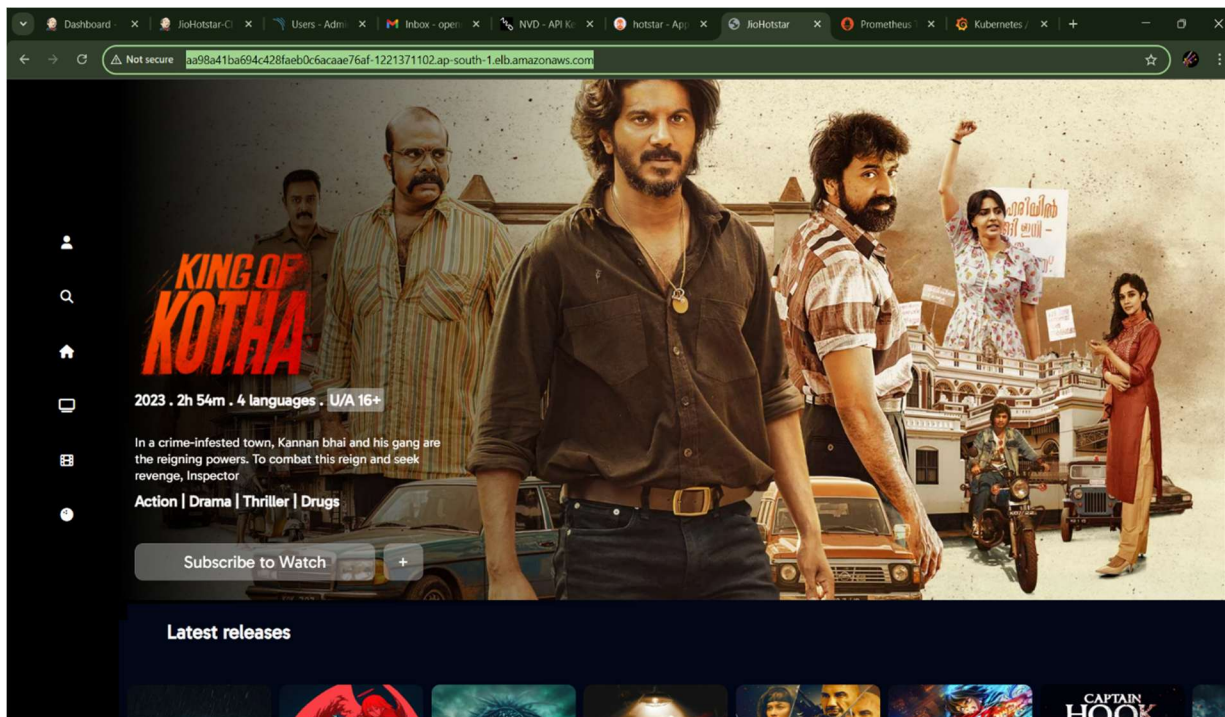
The ArgoCD interface shows the 'hotstar' application. The top bar includes navigation links: APP HEALTH, SYNC STATUS, SYNC, HISTORY AND R, and a 'To exit full screen, press and hold Esc' button. The main area displays the application details, including the 'LAST SYNC' section.

The 'LAST SYNC' section shows a successful sync to HEAD (703659f) at 2023-11-09 02:27:22 GMT+0530. The sync was successful 7 minutes ago, with the author 'Jenkins-jenkins@ip-172-31-8-114.ap-south-1.com...' and the comment 'Updated environment variables'.

The application diagram shows the following components:

- hotstar (Application)
- hotstar-service (Service)
- hotstar-deploy (Deployment)
- hotstar-deploy-788d4bb8c4 (Deployment)
- hotstar-deploy-7756b54d9c (Deployment)
- hotstar-deploy-788d4bb8c4-s... (Deployment)
- hotstar-deploy-788d4bb8c4-s... (Deployment)
- hotstar-deploy-788d4bb8c4-s... (Deployment)
- hotstar-deploy-788d4bb8c4-s... (Deployment)

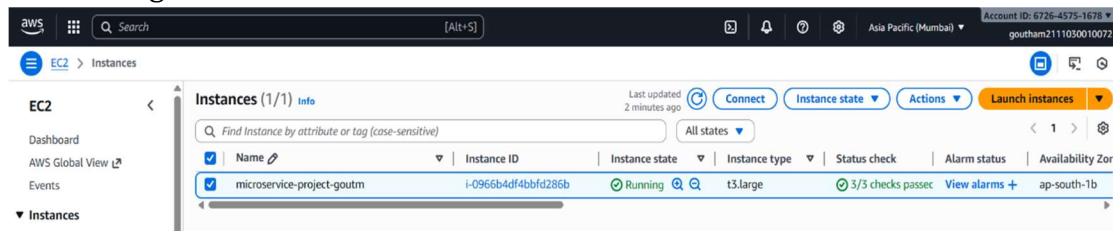
- Deployed Hotstar Clone



Pre-requisites to implement this project:

This project is implemented on Asia pacific Mumbai (ap-south-1).

- Create 1 Master machine (microservice-project-goutm) on AWS with 2CPU, 8GB of RAM (t3.large) and 30 GB of storage



- Open the below ports in security group of master machine

Inbound rules (9)							
Search							
☐	Name	Security group rule...	IP version	Type	Protocol	Port range	Source
☐	–	sgr-0bbaa8c2a68fe0a73	IPv4	Custom TCP	TCP	30000 - 32767	0.0.0.0/0
☐	–	sgr-08d4d0de0ad87c0...	IPv4	HTTP	TCP	80	0.0.0.0/0
☐	–	sgr-0251004d0311a6...	IPv4	Custom TCP	TCP	6379	0.0.0.0/0
☐	–	sgr-09251928c4f0593b1	IPv4	HTTPS	TCP	443	0.0.0.0/0
☐	–	sgr-0a958de1c5c35faf2	IPv4	SMTPS	TCP	465	0.0.0.0/0
☐	–	sgr-0e2259f634fb93315	IPv4	Custom TCP	TCP	3000 - 10000	0.0.0.0/0
☐	–	sgr-014c7ec6aa22ad3a0	IPv4	SSH	TCP	22	0.0.0.0/0
☐	–	sgr-0880097a4af79bead	IPv4	SMTP	TCP	25	0.0.0.0/0
☐	–	sgr-0b1b59fae0749df3d	IPv4	Custom TCP	TCP	6443	0.0.0.0/0

We are creating this master machine (microservice-project-goutm) because we will configure Jenkins, eksctl, EKS cluster creation from here.

Go to master machine (microservice-project-goutm) server and give the below commands :

1. Install & Configure Docker by using below commands

```
sudo apt-get update
```

```
sudo apt-get install docker.io -y  
sudo usermod -aG docker ubuntu && newgrp docker
```

To provide permission to docker socket so that docker build and push command do not fail give below command

```
sudo chmod 777 /var/run/docker.sock
```

```
root@ip-172-31-8-114:~/JioHotstar# apt-get install docker.io -y  
usermod -aG docker ubuntu && newgrp docker  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done
```

2. Install and configure Jenkins

```
sudo apt update -y  
sudo apt install fontconfig openjdk-17-jre -y
```

```
sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \  
https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
```

```
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \  
https://pkg.jenkins.io/debian-stable binary/ | sudo tee \  
/etc/apt/sources.list.d/jenkins.list > /dev/null
```

```
sudo apt-get update -y  
sudo apt-get install jenkins -y
```

```
root@ip-172-31-8-114:~/JioHotstar# vim jenkins.sh  
root@ip-172-31-8-114:~/JioHotstar# cat jenkins.sh  
apt update -y  
apt install fontconfig openjdk-17-jre -y  
  
wget -O /usr/share/keyrings/jenkins-keyring.asc \  
https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key  
  
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \  
https://pkg.jenkins.io/debian-stable binary/ | sudo tee \  
/etc/apt/sources.list.d/jenkins.list > /dev/null  
  
apt-get update -y  
apt-get install jenkins -y  
  
root@ip-172-31-8-114:~/JioHotstar# sh jenkins.sh
```

- Now, access Jenkins on the browser on port 8080 and configure it. (<public-ip>:8080)

3. Create EKS Cluster on AWS

- Create IAM user with **access keys and secret access keys**
- AWSCLI should be configured by using below commands

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
sudo apt install unzip
unzip awscliv2.zip
sudo ./aws/install
aws configure
```

```
root@ip-172-31-8-114:~/JioHotstar# vim awscli.sh
root@ip-172-31-8-114:~/JioHotstar# cat awscli.sh
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
sudo apt install unzip
unzip awscliv2.zip
sudo ./aws/install
aws configure
root@ip-172-31-8-114:~/JioHotstar# sh awscli.sh
```

```
You can now run: /usr/local/bin/aws --version
AWS Access Key ID [None]: 
AWS Secret Access Key [None]: 
Default region name [None]: ap-south-1
Default output format [None]: table
root@ip-172-31-8-114:~/JioHotstar#
```

- Install **kubect**

```
curl -o kubect https://amazon-eks.s3.us-west-2.amazonaws.com/1.19.6/2021-01-05/bin/linux/amd64/kubect
chmod +x ./kubect
sudo mv ./kubect /usr/local/bin
kubect version --short -client
```

```
root@ip-172-31-8-114:~/JioHotstar# vim kubect.sh
root@ip-172-31-8-114:~/JioHotstar# cat kubect.sh
curl -o kubect https://amazon-eks.s3.us-west-2.amazonaws.com/1.19.6/2021-01-05/bin/linux/amd64/kubect
chmod +x ./kubect
sudo mv ./kubect /usr/local/bin
kubect version --short --client

root@ip-172-31-8-114:~/JioHotstar# sh kubect.sh
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           %             %             Dload  Upload  Total   Spent    Left   Speed
100 57.4M  100 57.4M    0     0  9927k      0  0:00:05  0:00:05 --:--:-- 12.5M
Client Version: v1.19.6-eks-49a6c0
```

- Install **eksctl**

```
curl --silent --location
"https://github.com/weaveworks/eksctl/releases/latest/download/eksctl_$(uname -s)_amd64.tar.gz" | tar xz -C /tmp
sudo mv /tmp/eksctl /usr/local/bin
eksctl version
```

```
root@ip-172-31-8-114:~/JioHotstar# vim eksctl.sh
root@ip-172-31-8-114:~/JioHotstar# cat eksctl.sh
curl --silent --location "https://github.com/weaveworks/eksctl/releases/latest/download/eksctl_$(uname -s)_amd64.tar.gz" | tar xz -C /tmp
sudo mv /tmp/eksctl /usr/local/bin
eksctl version

root@ip-172-31-8-114:~/JioHotstar# sh eksctl.sh
0.216.0
```

- Create EKS Cluster using eksctl command

```
eksctl create cluster --name=hotstar-clone --version 1.30 --zones=ap-south-1a,ap-south-1b,ap-south-1c --without-nodegroup
```

```

root@ip-172-31-8-114:~/JioHotstar# eksctl create cluster --name=hotstar-clone --version 1.30 --zones=ap-south-1a,ap-south-1b,ap-south-1c --without-nodegroup
2025-11-08 18:10:59 [i] eksctl version 0.216.0
2025-11-08 18:10:59 [i] using region ap-south-1
2025-11-08 18:10:59 [i] subnets for ap-south-1a - public:192.168.0.0/19 private:192.168.96.0/19
2025-11-08 18:10:59 [i] subnets for ap-south-1b - public:192.168.32.0/19 private:192.168.128.0/19
2025-11-08 18:10:59 [i] subnets for ap-south-1c - public:192.168.64.0/19 private:192.168.160.0/19
2025-11-08 18:10:59 [i] Auto Mode will be enabled by default in an upcoming release of eksctl. This means managed node groups and managed networking add-ons will no longer be created by default. To maintain current behavior, explicitly set 'autoModeConfig.enabled: false' in your cluster configuration. Learn more: https://eksctl.io/usage/auto-mode/
2025-11-08 18:10:59 [i] using Kubernetes version 1.30
2025-11-08 18:10:59 [i] creating EKS cluster "hotstar-clone" in "ap-south-1" region with
2025-11-08 18:10:59 [i] if you encounter any issues, check CloudFormation console or try 'eksctl utils describe-stacks --region=ap-south-1 --cluster=hotstar-clone'
2025-11-08 18:10:59 [i] Kubernetes API endpoint access will use default of {publicAccess=true, privateAccess=false} for cluster "hotstar-clone" in "ap-south-1"
2025-11-08 18:10:59 [i] CloudWatch logging will not be enabled for cluster "hotstar-clone" in "ap-south-1"
2025-11-08 18:10:59 [i] you can enable it with 'eksctl utils update-cluster-logging --enable-types={SPECIFY-YOUR-LOG-TYPES-HERE (e.g. all)} --region=ap-south-1 --cluster=hotstar-clone

```

The screenshot shows the AWS Elastic Kubernetes Service (EKS) console. The 'Clusters (1)' section displays a table with the following data:

Cluster name	Status	Kubernetes version	Support period	Upgrade policy	Created	Provider
hotstar-clone	Active	1.30 Upgrade now	Extended support until July 23, 2026	Extended support	34 minutes ago	EKS

– Associate IAM OIDC Provider

eksctl utils associate-iam-oidc-provider --region ap-south-1 --cluster hotstar-clone --approve

```

root@ip-172-31-8-114:~/JioHotstar# eksctl utils associate-iam-oidc-provider --region ap-south-1 --cluster hotstar-clone --approve
2025-11-08 18:26:42 [i] will create IAM Open ID Connect provider for cluster "hotstar-clone" in "ap-south-1"
2025-11-08 18:26:42 [✓] created IAM Open ID Connect provider for cluster "hotstar-clone" in "ap-south-1"

```

– Create Nodegroup using eksctl command

eksctl create nodegroup --cluster=hotstar-clone --region=ap-south-1 --name=hotstar-clone-ng-1 --node-type=t2.large --nodes=2 --nodes-min=2 --nodes-max=3 --node-volume-size=29 --ssh-access --ssh-public-key=eks-nodegroup-key-mumbai --managed --asg-access --external-dns-access --full-ecr-access --appmesh-access --alb-ingress-access

```

root@ip-172-31-8-114:~/JioHotstar# eksctl create nodegroup --cluster=hotstar-clone --region=ap-south-1 --name=hotstar-clone-ng-1 --node-type=t3.large --nodes=2 --nodes-min=2 --nodes-max=3 --node-volume-size=30 --ssh-access --ssh-public-key=eks-nodegroup-key-mumbai --managed --asg-access --external-dns-access --full-ecr-access --appmesh-access --alb-ingress-access
2025-11-08 18:28:13 [i] will use version 1.30 for new nodegroup(s) based on control plane version
2025-11-08 18:28:14 [i] nodegroup "hotstar-clone-ng-1" will use "" [AmazonLinux2023/1.30]
2025-11-08 18:28:14 [i] using EC2 key pair "eks-nodegroup-key-mumbai"
2025-11-08 18:28:14 [i] 1 nodegroup (hotstar-clone-ng-1) was included (based on the include/exclude rules)
2025-11-08 18:28:14 [i] will create a CloudFormation stack for each of 1 managed nodegroups in cluster "hotstar-clone"

```

Make sure the ssh-public-key “eks-nodegroup-key-mumbai” is available in your aws account.

The screenshot shows the AWS console with the 'hotstar-clone' cluster selected. The 'Node groups (1)' section displays a table with the following data:

Group name	Desired size	AMI release version	Launch template	Status
hotstar-clone-ng-1	2	1.30.14-20251103	eksctl-hotstar-clone-nodegroup-hotstar-clone-ng-1 (1)	Active

Below the node groups, the 'Instances (3)' section displays a table with the following data:

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS
hotstar-clone-hotstar-clone-ng-1-Node	i-0cf89e80301b80072	Running	t3.large	3/3 checks passed	View alarms +	ap-south-1b	ec2-13-203-79-145.ap-...
microservice-project-goutm	i-0966b4df4bfdf286b	Running	t3.large	3/3 checks passed	View alarms +	ap-south-1b	ec2-13-126-251-3.ap-...
hotstar-clone-hotstar-clone-ng-1-Node	i-0b48d519a31f2a4fc	Running	t3.large	3/3 checks passed	View alarms +	ap-south-1c	ec2-3-111-46-252.ap-s...

4. Install and configure SonarQube (Master machine)

```
docker run -itd --name SonarQube-Server -p 9000:9000 sonarqube:lts-community
```

and give “docker ps -a” command to check if the container is up or not.

```
root@ip-172-31-8-114:~/JioHotstar# docker run -itd --name SonarQube-Server -p 9000:9000 sonarqube:lts-community
Unable to find image 'sonarqube:lts-community' locally
lts-community: Pulling from library/sonarqube
60d98d907669: Pull complete
e24a8b9e652f: Pull complete
f3929ce9ef98: Pull complete
1df735f481ad: Pull complete
5d5a1fad7028: Pull complete
eb27e3a98da1: Pull complete
c7ad1fe61e07: Pull complete
4f4fb700ef54: Pull complete
Digest: sha256:f709975ab31d2d08f5a3ae2dc73a31ee011afc8cf28845082c17c55d45df9df5
Status: Downloaded newer image for sonarqube:lts-community
b265fbd35e609ab511c25427432b95aeb353109237b94c3efec077380b33b7cd
root@ip-172-31-8-114:~/JioHotstar# docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
b265fbd35e60	sonarqube:lts-community	"/opt/sonarqube/dock..."	17 seconds ago	Up 15 seconds	0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp	SonarQube-Server

5. Install Trivy (Master machine)

```
sudo apt-get install wget apt-transport-https gnupg lsb-release -y
wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key | sudo apt-key
add -
echo deb https://aquasecurity.github.io/trivy-repo/deb $(lsb_release -sc) main | sudo
tee -a /etc/apt/sources.list.d/trivy.list
sudo apt-get update -y
sudo apt-get install trivy -y
```

```
root@ip-172-31-8-114:~/JioHotstar# vim trivy.sh
root@ip-172-31-8-114:~/JioHotstar# cat trivy.sh
wget https://github.com/aquasecurity/trivy/releases/download/v0.18.3/trivy_0.18.3_Linux-64bit.tar.gz
tar xzvf trivy_0.18.3_Linux-64bit.tar.gz
sudo mv trivy /usr/local/bin/
echo "export PATH=$PATH:/usr/local/bin/" >> .bashrc
source .bashrc
root@ip-172-31-8-114:~/JioHotstar# sh trivy.sh
```

6. Install Helm (Master Machine)

```
curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
chmod 700 get_helm.sh
./get_helm.sh
helm version
```

```
root@ip-172-31-8-114:~/JioHotstar# vim helm.sh
root@ip-172-31-8-114:~/JioHotstar# cat helm.sh
curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
chmod 700 get_helm.sh
./get_helm.sh
helm version
root@ip-172-31-8-114:~/JioHotstar# sh helm.sh
Downloading https://get.helm.sh/helm-v3.19.0-linux-amd64.tar.gz
Verifying checksum... Done.
Preparing to install helm into /usr/local/bin
helm installed into /usr/local/bin/helm
version.BuildInfo{Version:"v3.19.0", GitCommit:"3d8990f0836691f0229297773f3524598f46bda6", GitTreeState:"clean", GoVersion:"go1.24.7"}
```

7. Install argocd through Helm (Master Machine)

```
helm repo add argo-cd https://argoproj.github.io/argo-helm
helm repo update
kubectl create namespace argocd
helm install argocd argo-cd/argo-cd -n argocd
```

```

root@ip-172-31-8-114:~/JioHotstar# helm repo add argo-cd https://argoproj.github.io/argo-helm
"argo-cd" has been added to your repositories
root@ip-172-31-8-114:~/JioHotstar# helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "argo-cd" chart repository
Update Complete. *Happy Helming!*
root@ip-172-31-8-114:~/JioHotstar# kubectl create namespace argocd
namespace/argocd created
root@ip-172-31-8-114:~/JioHotstar# helm install argocd argo-cd/argo-cd -n argocd
NAME: argocd
LAST DEPLOYED: Sat Nov  8 19:39:51 2025
NAMESPACE: argocd
STATUS: deployed
REVISION: 1
TEST SUITE: None

```

- Check argocd services
`kubectl get all -n argocd`
- Change argocd server's service from ClusterIP to NodePort
`kubectl patch svc argocd-server -n argocd -p '{"spec": {"type": "LoadBalancer"}}'`
- Confirm service is patched or not
`kubectl get all -n argocd`

```

root@ip-172-31-8-114:~/JioHotstar# kubectl get all -n argocd
NAME                                READY   STATUS    RESTARTS   AGE
pod/argocd-application-controller-0  1/1     Running   0           110s
pod/argocd-applicationset-controller-9cf989944-cv725  1/1     Running   0           110s
pod/argocd-dex-server-5d9db97575-f5c9k  1/1     Running   0           110s
pod/argocd-notifications-controller-5957db98b8-c9rjf  1/1     Running   0           110s
pod/argocd-redis-6d7d86c86b-vm7g6  1/1     Running   0           110s
pod/argocd-repo-server-575b95996f-mf5l4  1/1     Running   0           110s
pod/argocd-server-8646c94698-2qk75  1/1     Running   0           110s

NAME                                TYPE                      CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
service/argocd-applicationset-controller  ClusterIP              10.100.130.236   <none>            7800/TCP          111s
service/argocd-dex-server                 ClusterIP              10.100.3.127     <none>            5556/TCP,5557/TCP 111s
service/argocd-redis                      ClusterIP              10.100.105.80    <none>            6379/TCP          111s
service/argocd-repo-server                 ClusterIP              10.100.63.177    <none>            8081/TCP          111s
service/argocd-server                     LoadBalancer          10.100.110.73    aa55b5964b78946c9a64849648e27e84-521930272.ap-south-1.elb.amazonaws.com  80:30154/TCP,443:30349/TCP 111s

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/argocd-applicationset-controller  1/1     1             1           110s
deployment.apps/argocd-dex-server                 1/1     1             1           110s
deployment.apps/argocd-notifications-controller  1/1     1             1           111s
deployment.apps/argocd-redis                      1/1     1             1           111s
deployment.apps/argocd-repo-server                 1/1     1             1           110s
deployment.apps/argocd-server                     1/1     1             1           110s

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/argocd-applicationset-controller  1          1          1       110s
replicaset.apps/argocd-dex-server-5d9db97575     1          1          1       110s
replicaset.apps/argocd-notifications-controller-5957db98b8  1          1          1       110s
replicaset.apps/argocd-redis-6d7d86c86b          1          1          1       110s
replicaset.apps/argocd-repo-server-575b95996f     1          1          1       110s
replicaset.apps/argocd-server-8646c94698         1          1          1       110s

NAME                                READY   AGE
statefulset.apps/argocd-application-controller  1/1     110s

```

- Access it on browser, click on advance and proceed with the load balancer dns.
- Fetch the initial password of argocd server

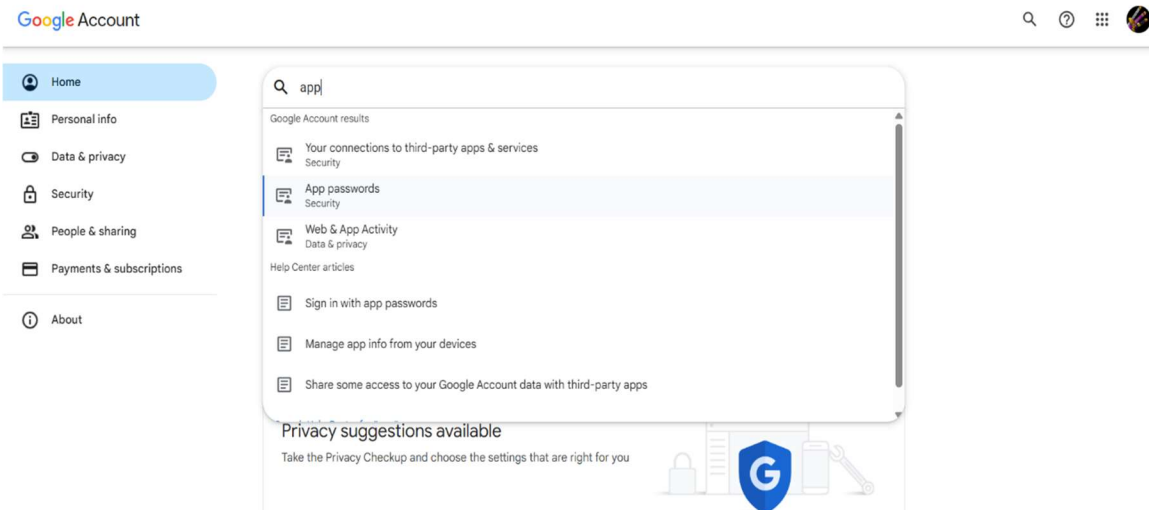
```

kubectl -n argocd get secret argocd-initial-admin-secret -o
jsonpath="{.data.password}" | base64 -d; echo

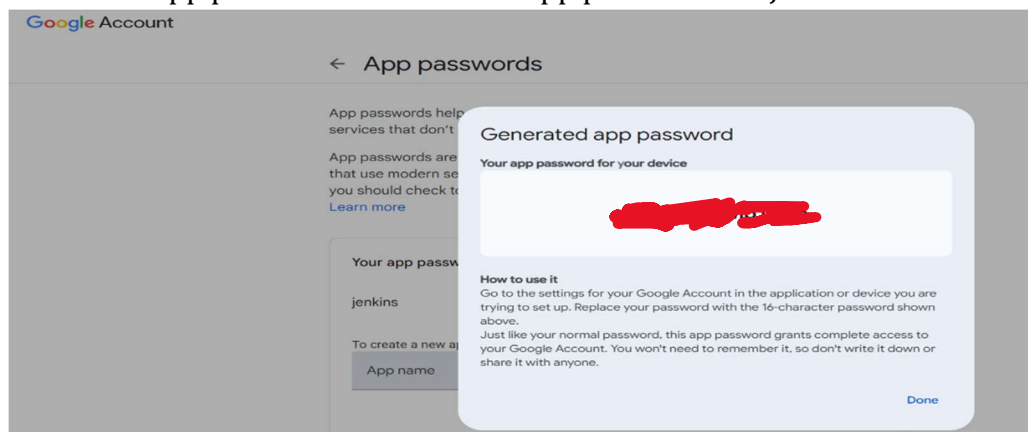
```

8. Steps to add email notification

- Go to your Jenkins Master EC2 instance and allow 465 port number for SMTPS
- Now, we need to generate an application password from our gmail account to authenticate with Jenkins
- Open gmail and go to Manage your Google Account -> Security > [!Important] > Make sure 2 step verification must be on



- Search for App password and create a app password for Jenkins



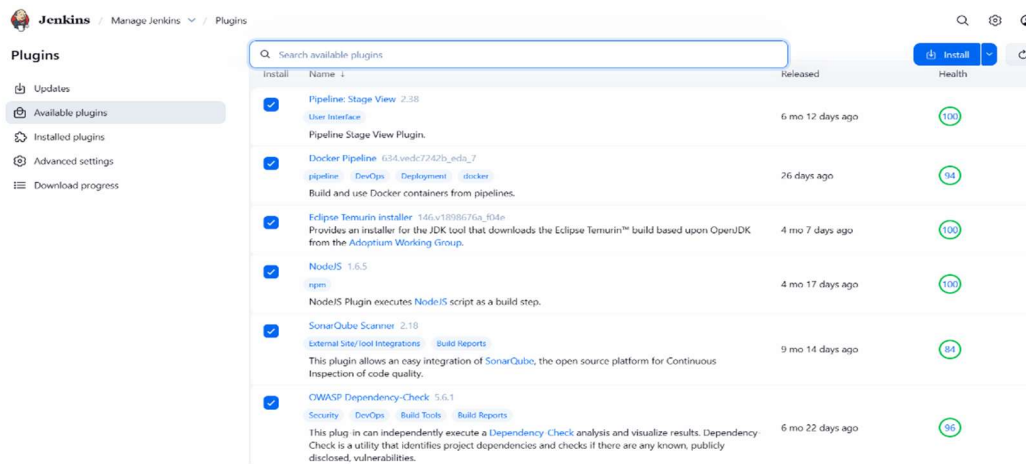
- Once, app password is create and go back to jenkins Manage Jenkins -> Credentials to add username and password for email notification



- Go back to Manage Jenkins -> System and search for Extended E-mail Notification
- Scroll down and search for E-mail Notification and setup email notification -> Enter your gmail and password which we copied recently in password field E-mail Notification -> Advance -> Save

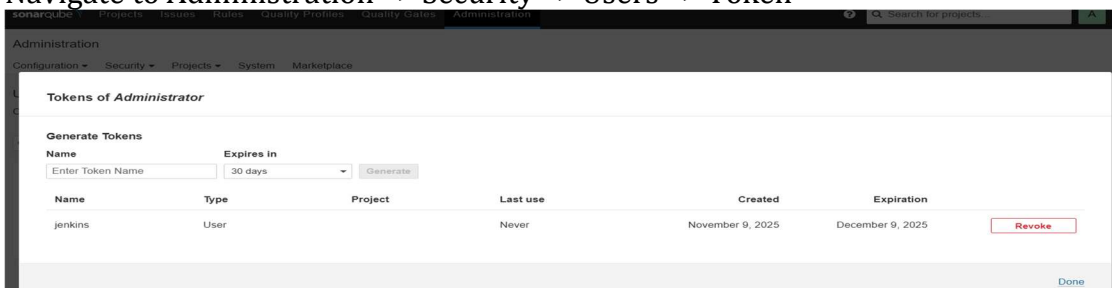
Steps to implement the project:

- Go to Jenkins Master and click on Manage Jenkins -> Plugins -> Available plugins install the below plugins:
 - Pipeline: Stage View
 - Docker Pipeline
 - Docker Eclipse Temurin Installer
 - NodeJS
 - SonarQube Scanner
 - OWASP Dependency-Check

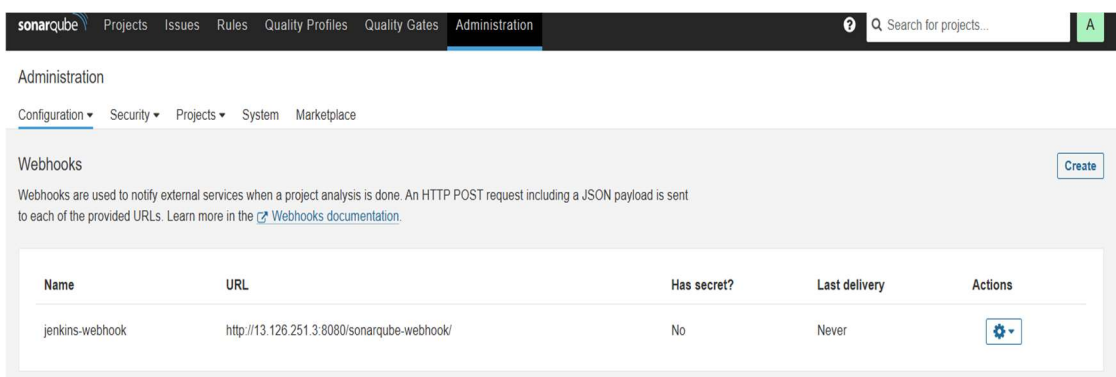


After installing restart the Jenkins

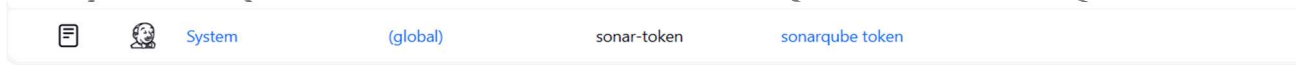
- Now move to Manage jenkins -> Tools
 - For NodeJs use version NodeJs 16.2.0
 - For Jdk17 use version jdk-17.0.8.1+1
 - For Dependency Check use latest version
 - For Sonar Scanner use latest version
- Login to SonarQube server and create the credentials and webhook for jenkins to integrate with SonarQube
 - Navigate to Administration -> Security -> Users -> Token



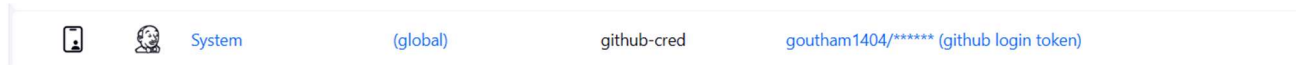
- Now go to Administration -> Webhook and click on create webhook to integrate with Jenkins



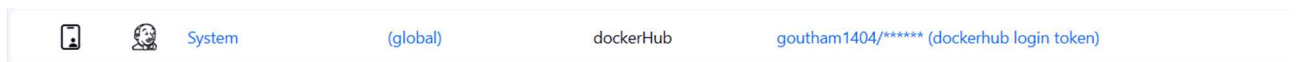
- Now, go to Manage Jenkins -> credentials and add Sonarqube credentials using kind as Secret text:



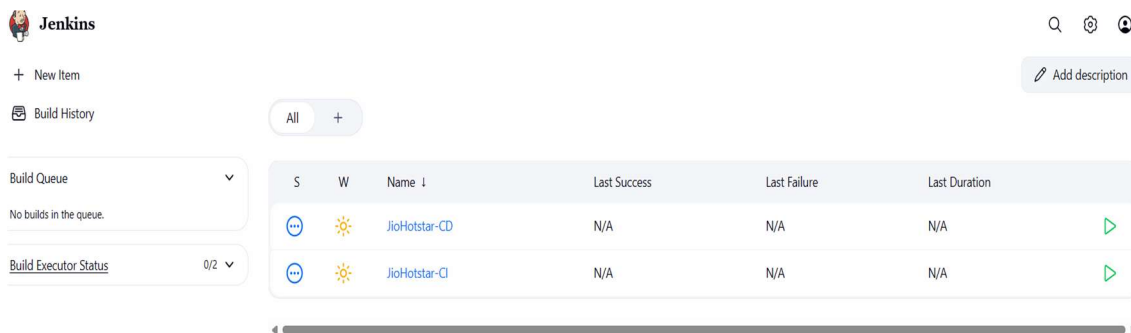
- Go to Manage Jenkins -> credentials and add Github credentials to push updated code from the pipeline:



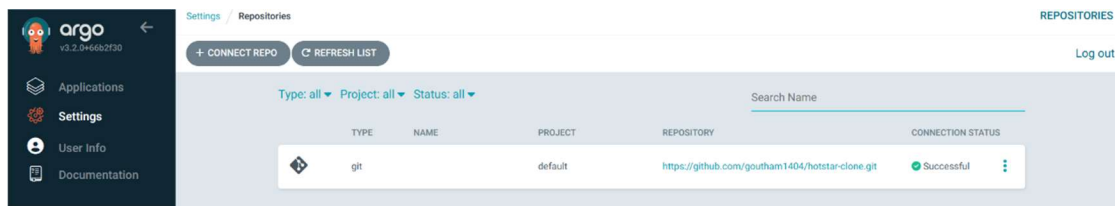
- While adding github credentials add Personal Access Token in the password field.
- Go to Manage Jenkins -> System and search for SonarQube installations:
- Add name -> Sonarqube server link (<public-ip>:9000) -> sonar-token (sonarqube credentials)
- Navigate to Manage Jenkins -> credentials and add credentials for docker login to push docker image:



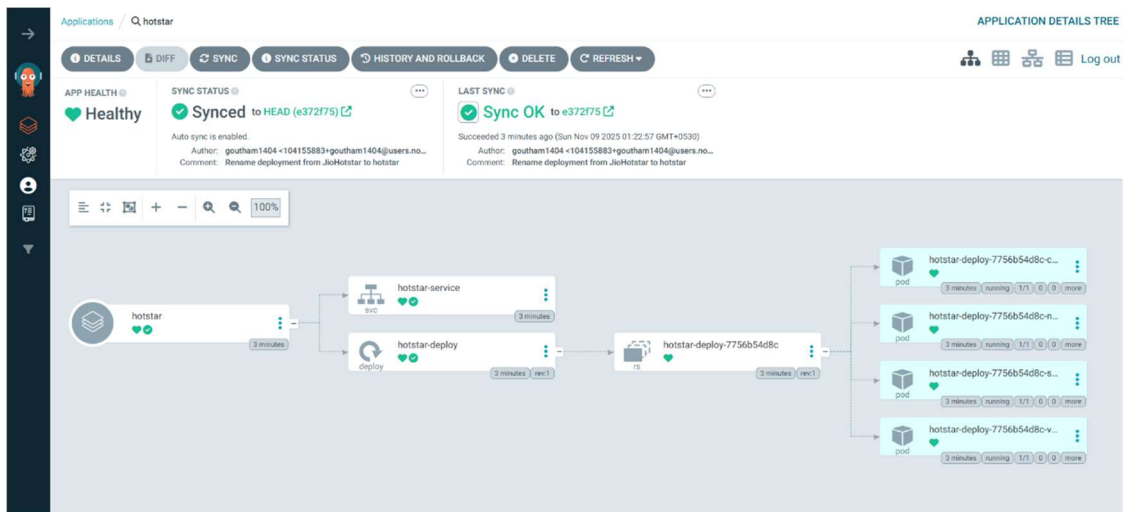
- While adding DockerHub credentials add Personal Access Token in the password field.
- Create a JioHotstar-CI pipeline and JioHotstar-CD pipeline
- For pipeline scripts go to <https://github.com/goutham1404/hotstar-clone.git>



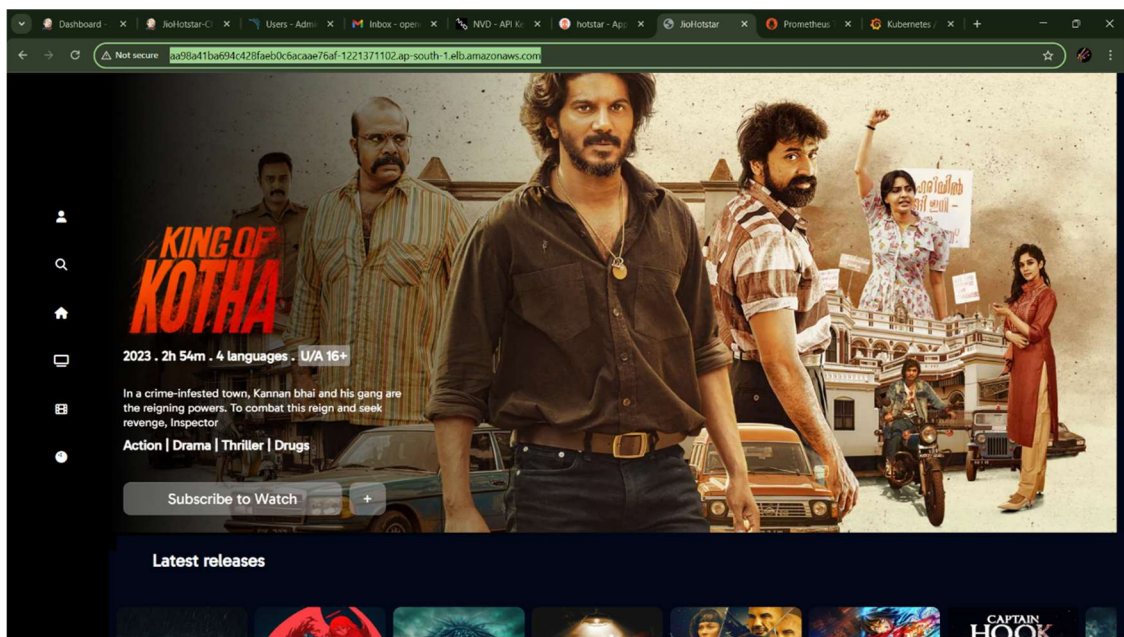
- Go to argoCD and connect your github repo
- Go to Settings -> Repositories -> connect repo



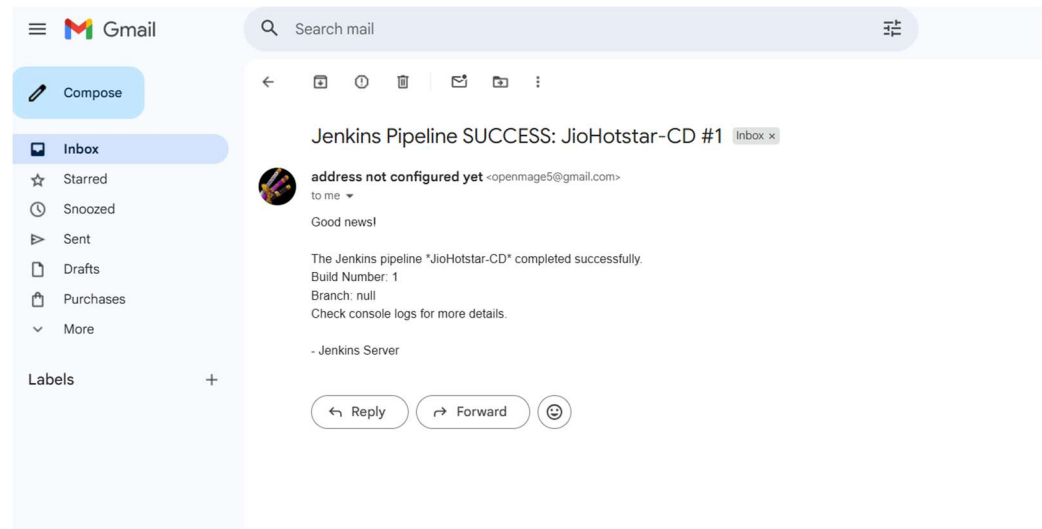
- Make sure that connection status is successful
- Now, go to Applications and click on New App your application is deployed on AWS EKS Cluster



- The deployed JioHotstar web application



- Email Notification



To monitor EKS cluster, kubernetes components and workloads using prometheus and grafana via HELM (On Master machine)

- Add Helm Stable Charts for Your Local Client

```
helm repo add stable https://charts.helm.sh/stable
```

- Add Prometheus Helm Repository

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
```

- Create Prometheus Namespace

```
kubectl create namespace prometheus
```

```
kubectl get ns
```

- Install Prometheus using Helm

```
helm install stable prometheus-community/kube-prometheus-stack -n prometheus
```

```
root@ip-172-31-8-114:~/JioHotstar# helm repo add stable https://charts.helm.sh/stable
"stable" has been added to your repositories
root@ip-172-31-8-114:~/JioHotstar# helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
"prometheus-community" has been added to your repositories
root@ip-172-31-8-114:~/JioHotstar# helm repo list
NAME                URL
argo-cd              https://argoproj.github.io/argo-helm
stable               https://charts.helm.sh/stable
prometheus-community https://prometheus-community.github.io/helm-charts
root@ip-172-31-8-114:~/JioHotstar# kubectl create namespace prometheus
namespace/prometheus created
root@ip-172-31-8-114:~/JioHotstar# helm install stable prometheus-community/kube-prometheus-stack -n prometheus
NAME: stable
LAST DEPLOYED: Sat Nov  8 20:34:11 2025
NAMESPACE: prometheus
STATUS: deployed
REVISION: 1
NOTES:
```

- Expose Prometheus and Grafana to the external world through Node Port and change it from Cluster IP to NodePort after changing make sure you save the file and open the assigned nodeport to the service.

```
kubectl edit svc stable-kube-prometheus-sta-prometheus -n Prometheus
```

- Verify service

```
kubectl get svc -n prometheus
```

- Now,let's change the SVC file of the Grafana and expose it to the outer world

```
kubectl edit svc stable-grafana -n prometheus
```

```

root@ip-172-31-8-114:~/JioHotstar# kubectl edit svc stable-kube-prometheus-sta-prometheus -n prometheus
service/stable-kube-prometheus-sta-prometheus edited
root@ip-172-31-8-114:~/JioHotstar# kubectl get svc -n prometheus
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)                                AGE
alertmanager-operated              ClusterIP           None            <none>           9093/TCP,9094/TCP,9094/UDP           107s
prometheus-operated                ClusterIP           None            <none>           9090/TCP                             106s
stable-grafana                     ClusterIP           10.100.127.255  <none>           80/TCP                               114s
stable-kube-prometheus-sta-alertmanager ClusterIP           10.100.221.146  <none>           9093/TCP,8080/TCP                   114s
stable-kube-prometheus-sta-operator ClusterIP           10.100.170.134  <none>           443/TCP                               114s
stable-kube-prometheus-sta-prometheus NodePort            10.100.84.227   <none>           9090:32676/TCP,8080:32053/TCP       114s
stable-kube-state-metrics           ClusterIP           10.100.157.180  <none>           8080/TCP                             114s
stable-prometheus-node-exporter     ClusterIP           10.100.67.33    <none>           9100/TCP                             114s
root@ip-172-31-8-114:~/JioHotstar# kubectl edit svc stable-grafana -n prometheus
service/stable-grafana edited
root@ip-172-31-8-114:~/JioHotstar# kubectl get svc -n prometheus
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)                                AGE
alertmanager-operated              ClusterIP           None            <none>           9093/TCP,9094/TCP,9094/UDP           5m10s
prometheus-operated                ClusterIP           None            <none>           9090/TCP                             5m9s
stable-grafana                     NodePort            10.100.127.255  <none>           80:31234/TCP                         5m17s
stable-kube-prometheus-sta-alertmanager ClusterIP           10.100.221.146  <none>           9093/TCP,8080/TCP                   5m17s
stable-kube-prometheus-sta-operator ClusterIP           10.100.170.134  <none>           443/TCP                               5m17s
stable-kube-prometheus-sta-prometheus NodePort            10.100.84.227   <none>           9090:32676/TCP,8080:32053/TCP       5m17s
stable-kube-state-metrics           ClusterIP           10.100.157.180  <none>           8080/TCP                             5m17s
stable-prometheus-node-exporter     ClusterIP           10.100.67.33    <none>           9100/TCP                             5m17s

```

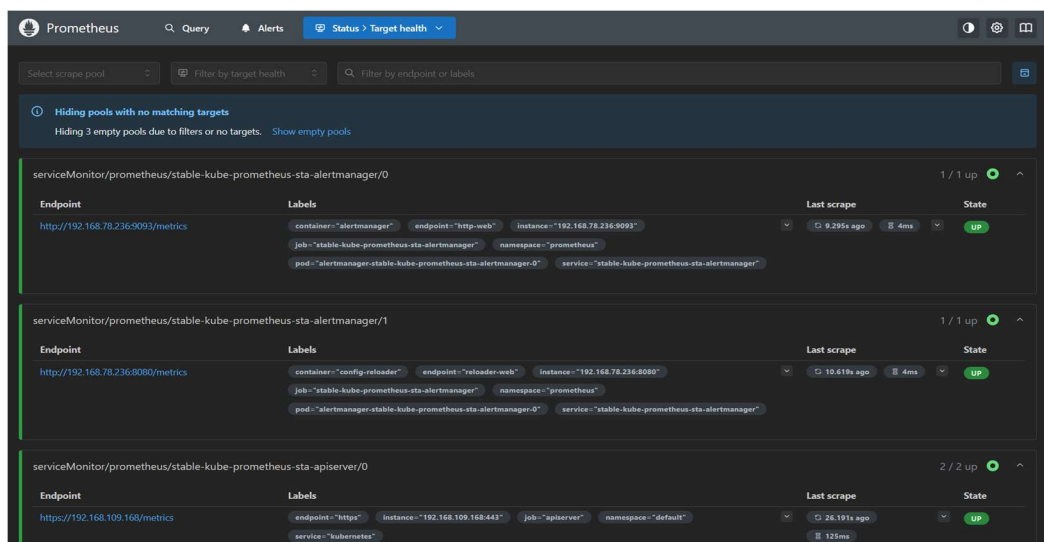
- Get a password for grafana

```

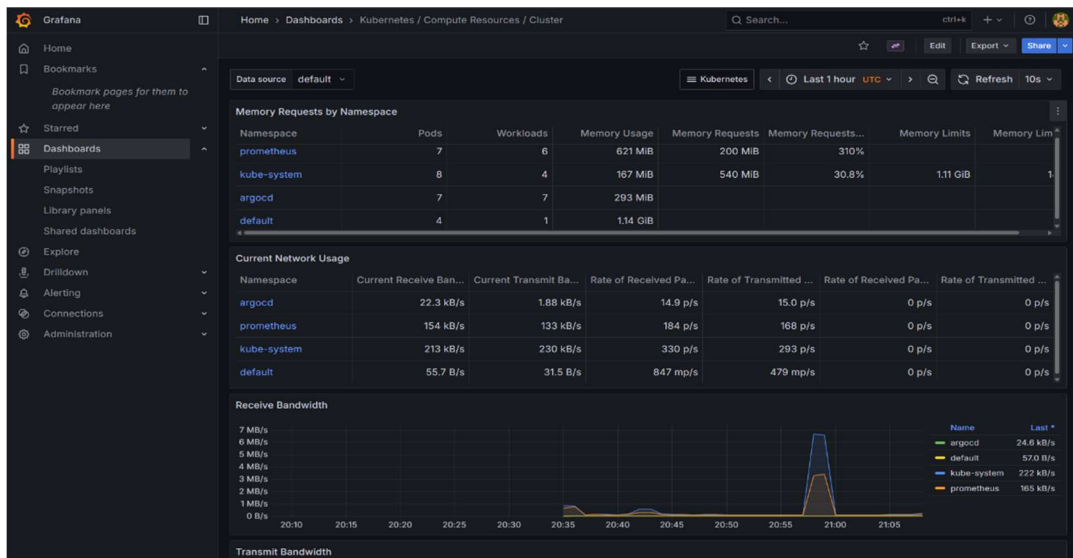
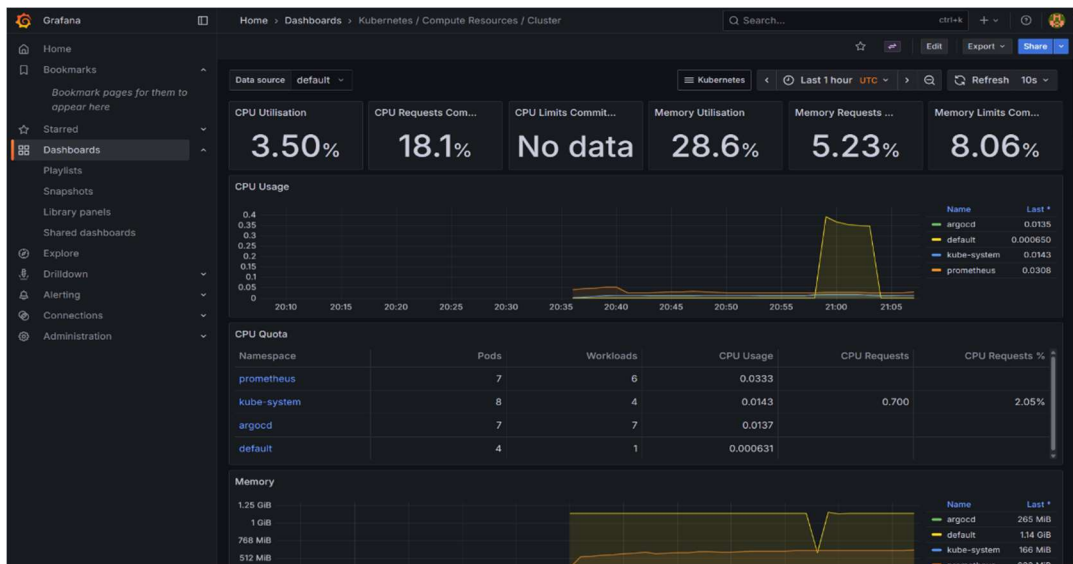
kubectl get secret --namespace prometheus stable-grafana -o jsonpath="{.data.admin-password}" | base64 --decode ; echo

```

- Now, view the dashboard in Prometheus



- Now, view the Dashboard in Grafana



To Clean Up EKS Cluster

- Delete eks cluster
eksctl delete cluster --name=hotstar-clone --region=ap-south-1
- To Delete eks cluster nodes
eksctl delete nodegroup --cluster hotstar-clone --name hotstar-clone-ng-1